

LAIR

B.LAIR_QT

(Booru) Local Archive & Indexing Repository

A desktop booru client. Qt6, C++, no Electron.

VERSION 1.0.0.1 · MPL 2.0 · re-v0

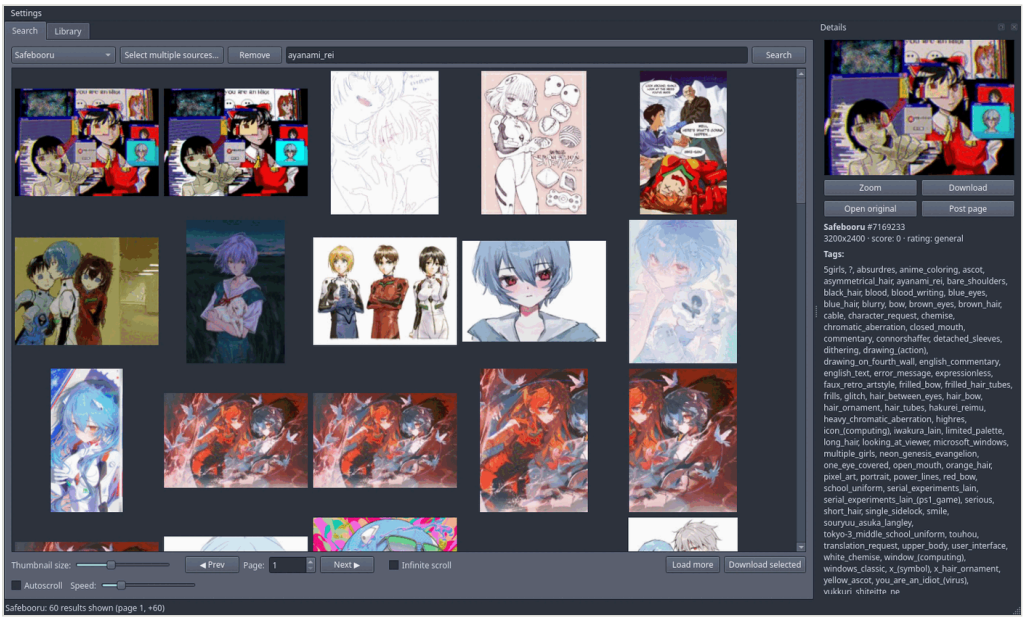
The name B.LAIR stands for (Booru) Local Archive & Indexing Repository. The `_QT` is not the toolkit — it stands for Cute, which happens to be how the toolkit is pronounced anyway.

B.LAIR_QT is native software for browsing boorus with a local gallery. What separates it from other booru clients is that the gallery acts as the booru repository itself: it respects source and tags, and lives in ordinary folders your file manager already understands.

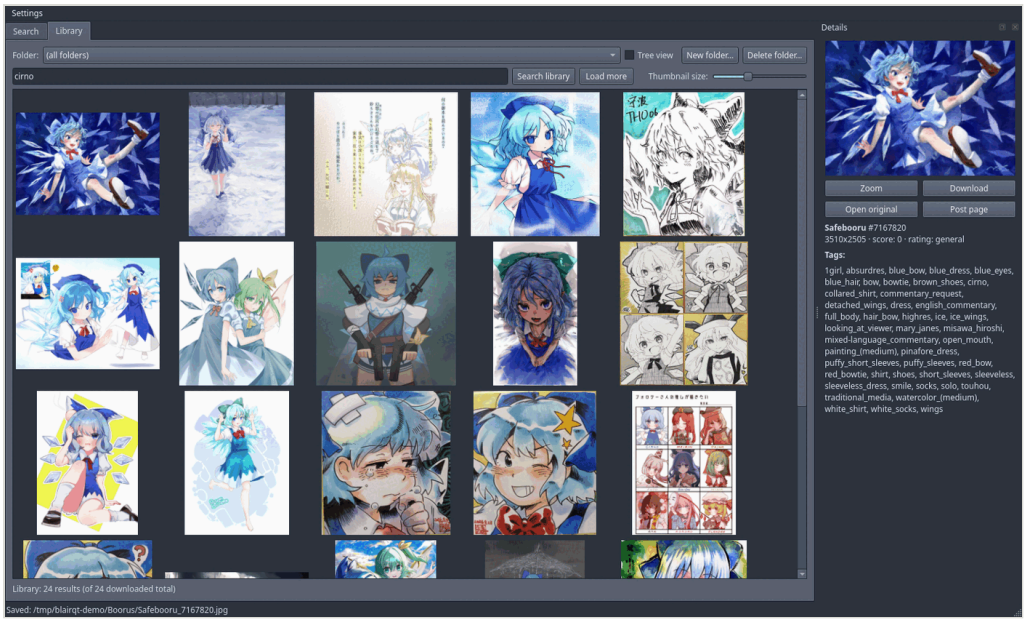
It searches several boorus at once and downloads the originals; tags go into a SQLite index next to the files. Delete the database and the files are still there, still sorted, still openable by anything else on the system.

Boorus that are not on the list are usually one URL away: the app probes the site against eight engines and picks the one that fits. Anything else takes a short Python plugin — or, for sites with no API at all, the bundled HTML-scrape bridge.

What it does not do: play sound, log into accounts, upload anything, or sandbox plugins.



Searching Safebooru; the details panel shows the clicked post.



The Library tab: downloaded posts, searchable by tag, in real folders.

Quickstart

Prebuilt packages are attached to each release on GitHub, **github.com/Re-V0/Booruqt/releases**. Download the one for your system — Ubuntu 24.04 and FreeBSD 14 have their own, ending in `-ubuntu24.04` and `-freebsd14` — then (with the version you downloaded):

```
sudo pacman -U blairqt-<version>-1-x86_64.pkg.tar.zst       # Arch
sudo apt install ./blairqt_<version>-debian13_amd64.deb     # Debian 13
sudo apt install ./blairqt_<version>-ubuntu26.04_amd64.deb  # Ubuntu 26.04
sudo pkg add blairqt-<version>-freebsd15-amd64.pkg          # FreeBSD 15
```

Windows (64-bit): unpack `blairqt-<version>-windows-x64.zip` anywhere. Once per computer, run `vc_redist.x64.exe` from that folder: it installs the Microsoft Visual C++ runtime, which the zip does not contain, and without it `blairqt.exe` does not start. Then run `blairqt.exe`. The program itself installs nothing; settings and the library index live in your user profile. `README.txt` in the zip says which runtime version the build needs.

macOS (Apple Silicon): open `blairqt-<version>-macos-arm64.dmg` and drag `blairqt` to Applications. The app is not signed with an Apple Developer ID, so the first start is blocked: open System Settings → Privacy & Security and click "Open Anyway" next to `blairqt`.

Python plugins need Python 3: `python3` or `python` on `PATH`, or on Windows the `py` launcher from python.org. Set `BLAIRQT_PYTHON` to an interpreter's full path to use a different one.

Arch Linux, from source. Build outside the source tree: makepkg keeps its sources in `./src`, the same name as this project's source folder, and `makepkg -c` would delete it.

```
mkdir -p ~/build/blairqt
cp PKGBUILD ~/build/blairqt/
cd ~/build/blairqt
makepkg -si
```

Gentoo — drop the ebuild into a local overlay:

```
mkdir -p /var/db/repos/local/media-gfx/blairqt
cp blairqt-1.0.0.1.ebuild /var/db/repos/local/media-gfx/blairqt/
ebuild /var/db/repos/local/media-gfx/blairqt/blairqt-1.0.0.1.ebuild manifest
emerge -av media-gfx/blairqt
```

`blairqt-9999.ebuild` builds the `main` branch instead. `USE=python` pulls an interpreter for script plugins; `USE=test` runs the suite during the merge.

Anything else — needs Qt 6 (Widgets, Network, Concurrent, Sql, Multimedia, LinguistTools), CMake 3.21+ and a C++20 compiler:

```
cmake -S . -B build -G Ninja -DCMAKE_BUILD_TYPE=Release
cmake --build build
./build/blairqt
```

On first start a welcome screen asks which sources to load. Nothing explicit is enabled unless you pick it. Type tags, hit Enter, drag the slider if the thumbnails are the wrong size. Right-click a post to download it or to blacklist a tag.

Sources

Eight generic parsers — Danbooru, Gelbooru/dapi, Moebooru, e621, Philomena, Szurubooru, Shimmie2, and Gelbooru 0.1 without an API (the booru.org boards, read from their HTML post list) — cover dozens of sites without a class per site.

Add custom booru works out which one fits from any address of the site: its front page, a post or a search, with or without `https://`. It reads the front page for the engine's traces and asks that engine first, then the others; it follows a redirect to `https` or `www.`, and keeps a booru installed in a subfolder. It spaces its requests out and stops at the first "429 Too Many Requests". When nothing fits it says why, if it can tell: a bot protection page (Cloudflare, Anubis, DDoS-Guard) in the way, an API that wants a key, too many requests, or a Gelbooru-engine site it cannot read, which the HTML-scrape bridge below may still manage.

Several sources can be merged into one gallery. Paging, infinite scroll and autoscroll are three ways to move through results; picking one disables the controls that stop making sense.

Sites behind Cloudflare or similar protection accept a cookie (such as `cf_clearance`) and the User-Agent of the browser it came from, pasted in when adding the source. They go with every request to that site, whatever its engine: searches, tag suggestions, thumbnails, downloads and the viewer. Sites

with no API at all can go through the bundled HTML-scrape bridge — those posts arrive without tags, rating or dimensions, which is the price of scraping HTML.

Every request, status code and response snippet lands in the network log dock. When a site misbehaves, that dock is the first place to look.

Video posts (WebM, MP4) play in the zoom viewer as soon as it opens, in a loop and without sound, and so do GIFs. **Settings → Appearance → Animated thumbnails** makes video and GIF posts move in the grid too. It is off by default, because each of them downloads its whole file: only posts in view play, six at most, none over 32 MB.

Library

Downloads are indexed in SQLite and written into real nested folders — the same folders Dolphin, Thunar, Explorer or Finder show. Move files between them from inside the app and the tags follow; re-downloading a post you already have is detected and skipped. A download is written to a temporary file and renamed only when complete, so a broken transfer never leaves half an image behind.

Each image also gets a hidden tags sidecar next to it — `image.png` gets `.image.png.txt`, one line of space-separated tags. It follows the file when you move it, and it is what lets another tool (or you, with `grep`) read the tags without touching the database. The metadata is stored as `.txt` as well, because why not — txt is cheap.

Search the library by tag, filter by folder, with or without subfolders. Tags match whole, ignoring case: `solo` does not find `solo_focus`. Zoom opens local files instantly, animated GIFs and video included.

Blacklist

A tag blacklist, global or per source, editable in Settings or straight from a post's context menu. Sites that understand `-tag` get the exclusions pushed into the query; the rest are filtered before they reach the grid.

Matching is exact: `solo` does not hide `solo_focus`. It can apply to the library too, if you want the same rules on what you already downloaded.

Plugins

A new booru engine is one Python file. No C++, no rebuild:

```
def build_search_request(base_url, tags, page, per_page) -> dict:
    """Return {"url": ..., "headers": {...}}."""

def parse_response(body, base_url) -> list[dict]:
    """Return post dicts: id, file_url, preview_url, sample_url,
    post_page_url, tags (list), rating, score, width, height."""

def supports_tag_exclusion() -> bool:      # optional
    """True if the site understands "-tag", so the blacklist can be
    applied in the query instead of only filtering the results."""
```

Add it through **Add custom booru** → *Plugin script*; with Auto-detect on, the script is tried automatically. Tracebacks show up in the dialog log and the network log dock. Plugins need Python 3, found as described under Quickstart, and some need their own modules — the Newgrounds example in `examples/` wants `argon2-cffi`.

> **Plugins run with your full user permissions. Only add scripts you trust.**

HTML-scrape bridge

For Gelbooru-engine sites with no `dapi`. Not started automatically:

```
pip install flask cloudscraper beautifulsoup4
python3 tools/html_scrape_bridge.py
```

It listens on `127.0.0.1:8765`. Tick *This site has no API (dapi)* when adding the source. The packages ship the script too: `/usr/share/blairqt/tools/` on Linux, `/usr/local/share/blairqt/tools/` on FreeBSD, `tools\` next to `blairqt.exe` on Windows, `blairqt.app/Contents/Resources/tools/` on macOS. The tooltip of that checkbox shows the full path.

Appearance and language

Themes are JSON over `QPalette`; 20 curated ones ship with the app and the system default is one click away. Interface languages: Polish, English, German, Danish, Russian, following the system language when it is one of those. Window and dock layout survive a restart.

Safety

The app writes inside its own data folder and folders you explicitly grant, and nowhere else. That is application discipline, not a sandbox — a plugin you add runs as you do. Stored API keys and cookies (`custom_sources.json`) are written owner-only (0600).

Tests

No network, no visible window:

```
ctest --test-dir build --output-on-failure
```

They cover blacklist matching, tag input, response parsing for every engine, source detection (against a local server playing each engine), the library (whole-tag search, paging, moving files, upgrading an older database), downloads, thumbnail loading, themes, the source list, and the rule that a plugin never blocks the GUI thread.

Data locations

Linux and FreeBSD

- Settings: `~/.config/blairqt/blairqt.conf`
- Library DB, sources, theme: `~/.local/share/blairqt/blairqt/`
- Thumbnail cache: `~/.cache/blairqt/blairqt/blairqt/thumbnails/`
- Downloads, until you pick another folder: `blairqt` in your pictures folder (`XDG_PICTURES_DIR`, usually `~/Pictures`)

Windows

- Settings: registry, `HKEY_CURRENT_USER\Software\blairqt\blairqt`
- Library DB, sources, theme: `%APPDATA%\blairqt\blairqt\`
- Thumbnail cache: `%LOCALAPPDATA%\blairqt\blairqt\cache\blairqt\thumbnails\`
- Downloads: `blairqt` in your Pictures folder

macOS

- Settings: `~/Library/Preferences/com.blairqt.blairqt.plist`
- Library DB, sources, theme: `~/Library/Application Support/blairqt/blairqt/`
- Thumbnail cache: `~/Library/Caches/blairqt/blairqt/blairqt/thumbnails/`
- Downloads: `~/Pictures/blairqt/`

Contributing

Issues and merge requests are welcome. Before sending a patch, run the tests — they take a second and need no network and no display:

```
ctest --test-dir build --output-on-failure
```

Windows and macOS

Every push to `main` builds, tests and packages the program for all systems listed under Quickstart, Windows and macOS included. I do not run either of those two myself, so reports from them are especially welcome: say which version of Windows or macOS, and attach the network log if a site is involved.

Support

Buy me a Linux license — the evil GNU charges me $70 per install: **buymeacoffee.com/re_v0**

License

Copyright (c) 2026 re-v0. Mozilla Public License 2.0 — see the `LICENSE` file in the source tree, which is the unmodified Mozilla text.

MPL copyleft is per file: you may combine these files with code under another license, but modifications to the files themselves stay MPL and stay open.

Trademarks. Qt is a registered trademark of The Qt Company Ltd. B.LAIR_QT is an independent project and is neither affiliated with nor endorsed by The Qt Company. It links the Qt 6 libraries dynamically under the LGPL-3.0. The Linux and FreeBSD packages use the Qt your system provides. The Windows zip and the macOS app carry unmodified Qt libraries next to the program; replacing them with your own build of the same Qt version is allowed. Both carry the license texts (MPL-2.0, LGPL-3.0, GPL-3.0) in a `licenses` folder, with a `NOTICE.txt` naming the Qt version, where its source code is, and the FFmpeg libraries that come with it to play video.
